🎭 CSS Pseudo-classes Guide

Understanding Pseudo-classes

CSS pseudo-classes are keywords added to selectors that define a special state of an element. They allow you to style elements based on user interaction, position in the document tree, or other conditions without adding extra HTML elements or classes. Pseudo-classes start with a single colon (:).

Key Point: Pseudo-classes represent element states or relationships (e.g., :hover, :focus, :first-child). They differ from pseudo-elements (::before, ::after) which create new content. Pseudo-classes are applied dynamically based on user interaction or document structure, not hardcoded in HTML.

Common Pseudo-classes Reference

Pseudo-class	Description	Triggers
:hover	Applies when user hovers over element	Mouse pointer over element
:focus	Applies when element has keyboard focus	Tab key or click on form element
:active	Applies while element is being activated	Mouse button pressed
:visited	Applies to visited links	Link previously clicked
:link	Applies to unvisited links	Link not yet visited
:first-child	Selects element if it's first child of parent	DOM position
:last-child	Selects element if it's last child of parent	DOM position
:nth-child(n)	Selects nth child of parent	DOM position (1-indexed)
:nth-of-type(n)	Selects nth element of same type	DOM position by element type
:disabled	Applies to disabled form elements	HTML disabled attribute
:enabled	Applies to enabled form elements	No disabled attribute
:checked	Applies to checked checkboxes/radios	User selection
:not(selector)	Applies to elements NOT matching selector	Negation matching
:empty	Applies to elements with no children	DOM content

Practical Examples

Example 1: :hover - Interactive Button

```
/* CSS */ .button { background: #3498db; color: white; padding: 12px 20px; border: none; border-radius: 4px; cursor: pointer; transition: all 0.3s ease; } .button:hover { background: #2980b9; transform: translateY(-2px); box-shadow: 0 4px 8px rgba(0,0,0,0.2); }
```

HTML Usage:

```
<button class="button">Hover me!</button>
```

Live Demo:

Hover me!

Try hovering over the button to see the effect

Example 2: :focus - Form Input Focus

```
/* CSS */ .input { padding: 10px; border: 2px solid #bdc3c7; border-radius: 4px; transition: border-color 0.3s ease; } .input:focus { border-color: #667eea; outline: none; box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1); }
```

HTML Usage:

```
<input type="text" class="input" placeholder="Click me">
```

Live Demo:

Click me to see focus effect

Example 3: :active - Button Press Effect

```
/* CSS */ .button:active { transform: translateY(0); box-shadow: inset 0 2px 4px rgba(0,0,0,0.2); }
```

HTML Usage:

```
<button class="button">Click me!</button>
```

Live Demo:

Click and hold me!

Press and hold to see the active state

Example 4: :visited & :link - Link States

```
/* CSS */ a:link { color: #3498db; text-decoration: none; } a:visited { color: #8e44ad; } a:hover { border-bottom: 2px solid #3498db; }
```

HTML Usage:

```
<a href="https://example.com" class="link">Visit Example</a>
```

Live Demo:

Visit Example

Blue = unvisited, Purple = visited

Example 5: :first-child & :last-child

```
/* CSS */ li:first-child { background: #667eea; color: white; font-weight: bold; } li:last-child { border-bottom: none; }
```

HTML Usage:

```
<ul>
    <li>First Item</li>
    <li>Middle Item</li>
    <li>Last Item</li>
</ul>
```

Live Demo:

First Item (styled with :first-child)

Middle Item

Last Item (no border with :last-child)

Example 6: :nth-child(n) - Select by Position

```
/* CSS - Style every 2nd item */ li:nth-child(2n) { background: #f0f0f0; } li:nth-child(3) { background: #e74c3c; color: white; }
```

HTML Usage:

```
<ul>
    <li>Item 1</li>
    <li>Item 2 (2n)</li>
    <li>Item 3 (highlighted)</li>
    <li>Item 4 (2n)</li>
</ul>
```

Live Demo:

Item 2 (2n)

Item 3 (highlighted)

Item 4 (2n)

Example 7: :disabled & :enabled - Form States

```
/* CSS */ input:disabled { background: #ecf0f1; cursor: not-allowed; color: #7f8c8d; opacity: 0.6; } input:enabled { background: white; }
```

HTML Usage:

```
<input type="text" placeholder="Enabled">
<input type="text" placeholder="Disabled" disabled>
```

Live Demo:

Enabled input | Disabled input

Example 8: :checked - Checkbox/Radio Selection

```
/* CSS */ input[type="checkbox"]:checked + label { color: #27ae60; font-weight: bold; }
```

HTML Usage:

```
<input type="checkbox" id="agree">
<label for="agree">I agree to terms</label>
```

Live Demo:

☐ I agree to terms (styles green when checked)

Example 9: :nth-of-type - Select by Element Type

```
/* CSS - Style specific paragraph */ p:nth-of-type(2) { background: #667eea; color: white; padding: 15px; border-radius: 4px; }
```

HTML Usage:

```
<p>Paragraph 1</p>
<h3>Heading</h3>
<p>Paragraph 2 (2nd p element) - SELECTED</p>
```

Live Demo:

Paragraph 1

Heading (not a paragraph)

Paragraph 2 (2nd p element) - SELECTED

Example 10: :not(selector) - Negation

```
/* CSS - Style all p except first */ p:not(:first-child) { margin-top: 20px; border-top: 2px solid #ddd; padding-top: 15px; }
```

HTML Usage:

```
<p>First paragraph (no border)</p>
<p>Second paragraph (with border)</p>
```

Live Demo:

First paragraph (no border)

Second paragraph (with border from :not(:first-child))

Example 11: :empty - Empty Elements

```
/* CSS - Hide empty paragraphs */ p:empty { display: none; } div:empty { border: 2px dashed #95a5a6; height: 50px; display: flex; align-items: center; justify-content: center; color: #95a5a6; }
```

HTML Usage:

```
<p></p> (hidden by :empty)
<div></div> (shows dashed border)
<p>Content</p> (visible)
```

Live Demo:

Empty div

Paragraph with content (visible)

Pseudo-class Categories

Category	Pseudo-classes	Purpose
User Interaction	:hover, :focus, :active	Style based on mouse/keyboard interaction
Link States	:link, :visited	Distinguish unvisited and visited links
Structural	:first-child, :last-child, :nth-child()	Target elements by position in DOM
Form States	:disabled, :enabled, :checked	Style form elements based on state
Negation	:not()	Select elements NOT matching a selector

Best Practices

1. Enhance Accessibility: Always use :focus for keyboard navigation. Don't remove focus indicators - style them instead.

2. Maintain Visual Hierarchy: Use :hover, :focus, :active consistently. Users expect similar elements to behave similarly.

3. Use :nth-child Carefully: Remember :nth-child matches position, not type. Use :nth-of-type for specific element types.

4. Consider Mobile Users: :hover doesn't work on touch devices. Provide alternative interactions for mobile (e.g., tap states).

5. Performance with Complex Selectors: Avoid deeply nested :nth-child selectors. Use classes for frequently styled patterns.

Common Issues & Solutions

Issue 1: :focus Outline Hard to See
Problem: Default focus outline is hard to see
Solution: Style :focus instead of removing it. Add clear outline or box-shadow for better visibility.

Issue 2: :nth-child Selects Wrong Elements
Problem: :nth-child(n) matches position, not element type
Solution: Use :nth-of-type(n) instead for element-specific targeting.

Issue 3: :visited Not Working
Problem: Visited link colors don't apply
Solution: Security restriction - only color and background-color work with :visited. Can't change layout or size.

Advanced Pseudo-class Selectors

Selector	Description	Example
:nth-child(2n)	Every even child	li:nth-child(2n) - even list items
:nth-child(2n+1)	Every odd child	li:nth-child(2n+1) - odd list items
:nth-child(-n+3)	First 3 children	li:nth-child(-n+3) - first 3 items
:focus-within	Element or descendant has focus	form:focus-within
:valid	Form input passes validation	input:valid

Summary

CSS pseudo-classes are powerful tools for styling elements based on their state or position without modifying HTML. Understanding user interaction pseudo-classes (:hover, :focus, :active), structural pseudo-classes (:first-child, :nth-child), form states (:disabled, :checked), and negation (:not) enables you to create dynamic, responsive, and accessible user interfaces with pure CSS.

Key Takeaway: Master pseudo-classes for interactive styling and structural targeting. Always prioritize accessibility by maintaining focus states. Use :focus-visible for better keyboard navigation support. Remember that mobile devices don't support :hover, so provide alternative interactions.